
Error-Guessing

Testing by intuition and experience

Much ad hoc testing is based on intuition and guesswork and there are good reasons why this helps us to derive useful tests

Some people have a knack of finding bugs

Good test cases can be derived in this way

Error-guessed test cases should be documented as part of the test plan... in addition to the systematic testing approach.

Examples where intuitive testing can be effective

Some examples will give you an insight into how you might use error guessing to derive unusual scenarios to test.

A program reading a data file might be presented with:

wrong file, binary file, same file as output, no file

A screen that validates numbers might be given:

0,+0,-0,00.0, letter O, numbers with spaces, alphabetic or non-printing characters embedded, and so on.

Exploratory Testing

Intro to Exploratory Testing (ET)

Where the **documents** that form the **basis** for test design (e.g. requirements specification(s)) are of **very low quality or obsolete or do not exist at all**.

Perhaps just the program exists!

This technique may also be applicable when there is a **severe time restriction** (as it potentially uses far less time than other techniques)

Approach is mainly **based on the intuition and experience** of the tester

Intro to Exploratory Testing (ET)

A powerful and fun approach to testing

Most testers do exploratory testing at one time or another – it's just **not visible** to outsiders

No application of a **structured test process**

Possible **elements of the test object** (specific **tasks** and **functions**) are “**explored**”... leading to a decision regarding which parts to test.

Execute test cases, see if something “special” or unexpected occurs and run later tests to pursue this lead.

Gradually, knowledge about the test object is collected.

Intro to Exploratory Testing (ET)

Has a bad reputation in some circles

Sloppy, unprofessional, ineffective, unsystematic

Increasing credibility in the US and Europe

Test design and test execution at the same time

the opposite (?) of scripted testing (predefined test procedures, whether manual or automated).

Balancing ET with scripted testing

We become exploratory when

- the next test is influenced by the result of the last test

- we can't tell what tests should be run in advance

- we haven't had the opportunity to create those tests

If we are running scripted tests, and new information comes to light, we may switch to ET

We take a scripted approach when

- we know how we want to test

- new tests are relatively unimportant

- we need accuracy and repeatability in the tests

- we can pay for test documentation and maintenance.

Why do exploratory testing?

Scripted approach tries to document tester's ideas and there is a lot of value in that way of testing

But written test scripts disrupt the intellectual processes that help testers to find important problems quickly

If testing is intellectually rich and fluid, we will select better tests

The richness of ET is only limited by the breadth and depth of our imagination and our emerging insights into the nature of the product under test.

Mixing scripted tests and ET

Question the value of your tests, because no test process provides complete coverage

Maybe 10%, maybe 80% (no hard and fast rule) of each test cycle to exploratory testing

Pick one or more risk areas in the product and design and execute tests for that

Assign one tester to continuous ET duty – use them as a “scout” for new features.

Typical ET process

Briefing - one page charter to identify hotspots

Get familiar with product & target hotspots

- Try extremes, anything you like to break the product

- When you find a bug, explore around it

- Look for patterns, clues to other bugs

- Document the test that caused the failure; document other tests that cover 'interesting' situations

- Log an incident

- Move on to next interesting area

When to use ET? When normal testing is suspended or you believe more focused tests in an area will be productive.

ET and continuous Integration

Recall that continuous integration (CI) is the business of continually integration new (and usually small) code additions into the main code branch.

Typically, a fully fledged CI process will include a suite of regression tests to check that all pre-existing software is – to the extent that we can reasonably determine – unaffected by the newly added code.

With ET, some test cases may not be documented (and therefore not systematically repeatable).

Only the test cases that have identified issues *may* be documented / scripted / repeatable. This has the undesirable implication that many of the test performed in ET may not be easily integrated into the CI regression testing infrastructure.

Choosing Test Techniques

What influences the decision?

Format of requirements for black box techniques

Ranges of numbers – EP/BV

Classification of data into sets – EP, Classification Trees

... and so on

Poor test design basis documents – ET

Contractual requirements

One way for a customer to reduce risk and assure test thoroughness would be to mandate a level of testing

BS7925-2 was designed for this purpose – now superseded by
ISO/IEC/IEEE 29119-4

BS7925-2 defined auditable test process, design and measurement techniques

ISO/IEC/IEEE 29119-4:2015 defines test design techniques that can be used during the test design and implementation process that is defined in ISO/IEC/IEEE 29119-2.

It is quite reasonable for a customer to mandate, "the supplier must use boundary value analysis and branch test design techniques and achieve 100%"

Questions?

Any questions?